



종합 프로젝트

이미지 만들기부터 모니터링 확인까지, 두 개의 프로젝트로 완성합니다

프로젝트 세션

프로젝트 1 · 이미지 만들기 /
프로젝트 2 · 배포와 모니터링

프로젝트 진행 내용

리마인드 후 두 프로젝트를 순서대로 진행합니다

리마인드

Dockerfile 작성법, commit, push

프로젝트 1

본인 파일로 이미지를 Dockerfile과 commit, 두 가지 방법으로 만들고 Docker Hub에 올립니다.

프로젝트 2

교육 때 사용했던 예제를 kubectl 또는 Helm으로 배포하거나, 새로운 내용으로 배포하고, Prometheus-Grafana에서 직접 확인합니다.

PART 1

Docker 리마인드

Dockerfile 작성법, commit, push



Dockerfile 작성법

세 줄이면 충분한 예제로 다시 확인합니다

Dockerfile

```
# Dockerfile
FROM nginx:1.25
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

명령어	역할
FROM	어떤 이미지를 기반으로 시작할지
COPY	내 컴퓨터의 파일을 이미지 안으로 복사
EXPOSE	컨테이너가 사용할 포트를 문서화

docker build

Dockerfile로 이미지를 실제로 만듭니다

명령

```
# -t로 이미지 이름:태그를 지정, 마지막 점(.)은 현재 폴더  
docker build -t myapp:1.0 .
```

확인

```
docker images
```

확인 docker images 목록에 myapp:1.0이 보이면 성공입니다.

docker commit

실행 중인 컨테이너를 그대로 이미지로 굳힙니다

명령

```
# 1. 베이스 이미지로 컨테이너를 실행하고 내부로 접속
docker run -it --name temp-c nginx:1.25 bash
# 2. 컨테이너 내부에서 파일을 직접 수정 (컨테이너 안에서 실행)
echo "<h1>commit으로 만든 페이지</h1>" > /usr/share/nginx/html/index.html
# 3. 컨테이너에서 나온 뒤, 지금 상태를 이미지로 저장
exit
docker commit temp-c myapp-commit:1.0
```

차이 Dockerfile 없이도 이미지가 만들어집니다 — 다만 어떤 과정을 거쳤는지는 이미지 자체에 기록되지 않습니다

.

commit 활용 예시. 패키지 설치해서 굳히기

빈 OS 이미지에 필요한 도구를 설치하는, 더 흔한 시나리오입니다

명령

```
# 1. ubuntu 기본 이미지로 컨테이너 실행 및 내부 접속
docker run -it --name temp-ubuntu ubuntu:22.04 bash
# 2. 컨테이너 내부에서 원하는 패키지 설치 (컨테이너 안에서 실행)
apt-get update
apt-get install -y curl vim
# 3. 컨테이너에서 나온 뒤, 지금 상태를 이미지로 저장
exit
docker commit temp-ubuntu my-ubuntu-tools:1.0
```

연결 앞의 예제는 파일을 고쳤고, 이번엔 패키지를 설치했습니다 — 방식은 똑같이 "컨테이너 안에서 뭔가를 바꾸고, 나와서 commit" 입니다.

Docker Hub 준비

이미지를 올리려면 계정과 로그인이 필요합니다



Docker Hub = 이미지를 올려두는 공개 저장소

hub.docker.com에서 무료 계정을 만들고, VM 터미널에서 로그인하면 이미지를 올릴 준비가 끝납니다.

명령 (아이디·비밀번호 입력)

```
docker login
```

주의 무료 계정은 이미지가 기본적으로 공개(public)됩니다 — 민감한 내용은 넣지 마세요.

태그 붙여서 올리기

docker push 전에 이름 규칙을 맞춰야 합니다

명령

```
# 이미지 이름 앞에 본인 Docker Hub 아이디를 붙입니다
docker tag myapp:1.0 <본인아이디>/myapp:1.0
# 업로드
docker push <본인아이디>/myapp:1.0
```

핵심 Docker Hub는 <아이디>/<이미지이름>:<태그> 형식이 아니면 업로드를 거부합니다 — tag 명령으로 이름을 먼저 맞춰야 합니다.

Dockerfile vs commit, 언제 무엇을 쓸까

실무에서는 대부분 Dockerfile을 씁니다

	Dockerfile	commit
재현 가능성	동일한 이미지를 언제든지 다시 생성	그 순간의 상태만 저장
기록/버전관리	파일이라 Git으로 관리 가능	과정이 남지 않음
적합한 상황	정식 배포용 이미지	빠른 실험, 디버깅용 스냅샷

프로젝트 1

이미지 만들고 Docker Hub에 올리기

두 가지 방법으로 만들고, 최소 하나는 올립니다



프로젝트 1 안내

오늘 만들 결과물을 확인합니다



1. 파일 준비

본인이 원하는 파일(웹페이지 등)을 준비하거나, 예제를 그대로 사용합니다.



2. 두 가지 방법으로 이미지 생성

Dockerfile로 한 번, docker commit으로 한 번 — 같은 내용을 두 방법으로 만들어봅니다.



3. Docker Hub에 올리기

최소한 Dockerfile로 만든 이미지는 반드시 push합니다 — 프로젝트2에서 그대로 사용합니다.

실습. Dockerfile로 이미지 만들기

Part 1에서 본 구조를 그대로 적용합니다

Dockerfile

```
# Dockerfile (index.html과 같은 폴더에 저장)
FROM nginx:1.25
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

빌드 및 로컬 확인

```
docker build -t myapp:1.0 .
docker run -d -p 8080:80 --name myapp-c myapp:1.0
```

확인 curl http://localhost:8080으로 내용이 보이는지 확인하세요.

실습. commit으로도 만들어보기

같은 결과를 다른 방법으로 한 번 더 만듭니다

명령

```
# 1. 베이스 이미지로 컨테이너 실행 후 내부 접속
docker run -it --name temp-c2 nginx:1.25 bash
# 2. 컨테이너 내부에서 같은 내용으로 파일 작성
echo "<h1>안녕하세요, 제 프로젝트 이미지입니다</h1>" > /usr/share/nginx/html/index.html
# 3. 나와서 이미지로 저장
exit
docker commit temp-c2 myapp-commit:1.0
```

비교 docker images로 myapp:1.0과 myapp-commit:1.0을 나란히 확인하세요 — 결과 화면은 같지만 만든 과정이 다릅니다.

실습. Docker Hub에 올리기

Dockerfile로 만든 이미지를 기준으로 올립니다

명령

```
# 이름 맞추고 업로드
docker tag myapp:1.0 <본인아이디>/myapp:1.0
docker push <본인아이디>/myapp:1.0
```

확인 완료 후 브라우저에서 hub.docker.com/r/<본인아이디>/myapp 으로 접속해 이미지가 올라갔는지 확인하세요.

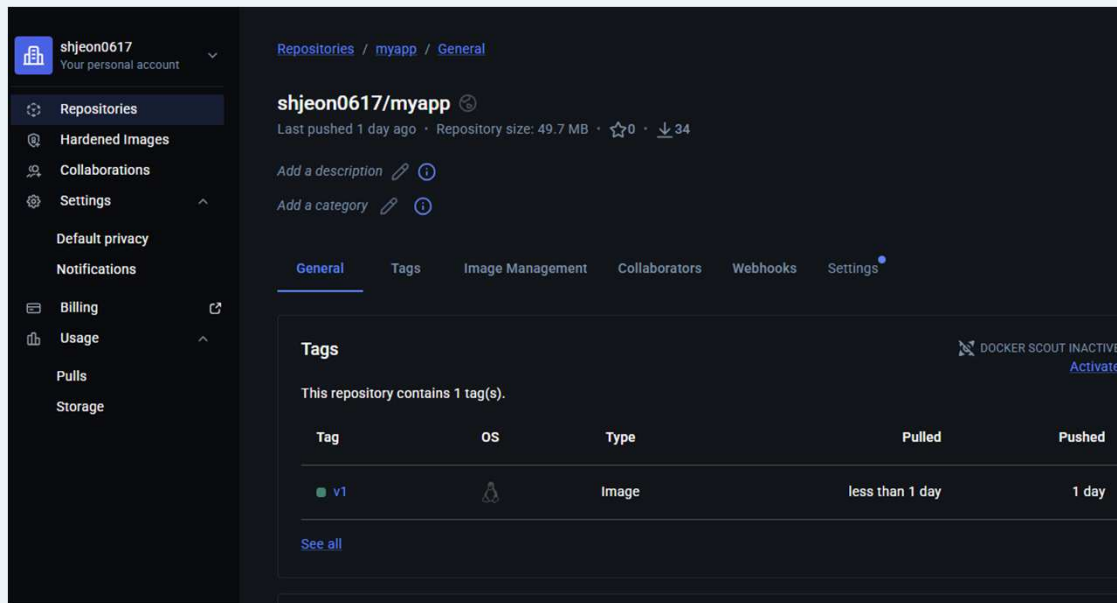
Docker Hub 캡처하기

프로젝트 1의 결과물



hub.docker.com 리포지토리 페이지를 캡처하세요

본인 아이디, 이미지 이름(myapp), 태그(1.0)가 화면에 함께 보이도록 캡처합니다.



프로젝트 2

배포하고 모니터링에서 확인하기

kubectl 또는 Helm, 편한 방법을 고르세요



프로젝트 2 안내

배포 방법을 하나 고르세요



kubectl 경로

Deployment YAML을 직접 작성해서 apply 합니다 — 가장 기본적인 방법입니다.



Helm 경로

helm create로 만든 Chart, add repo 로 추가한 Chart를 install 합니다.



공통. 모니터링 확인

어느 방법을 택하든, 배포 후에는 Prometheus·Grafana에서 똑같이 확인합니다.

실습. kubectl top으로 먼저 확인하기

모니터링 화면으로 가기 전, 순간 값부터 봅니다

명령

```
kubectl top pod
```

복습 이 명령이 값을 안 주면 metrics-server가 꺼져 있는 것입니다 — minikube addons enable metrics-server로 다시 켜세요.

실습. Prometheus에서 내 Pod 조회하기

Graph 탭에서 이름으로 필터링합니다

PromQL

```
# pod 이름에 myapp이 포함된 것만 조회  
container_cpu_usage_seconds_total{namespace="default", pod=~".*myapp.*"}
```

확인 kubectl port-forward로 Prometheus(9090)에 접속한 뒤 이 쿼리를 입력하세요. 결과가 나오면 Execute → Graph 탭에서 그래프로도 확인하세요.

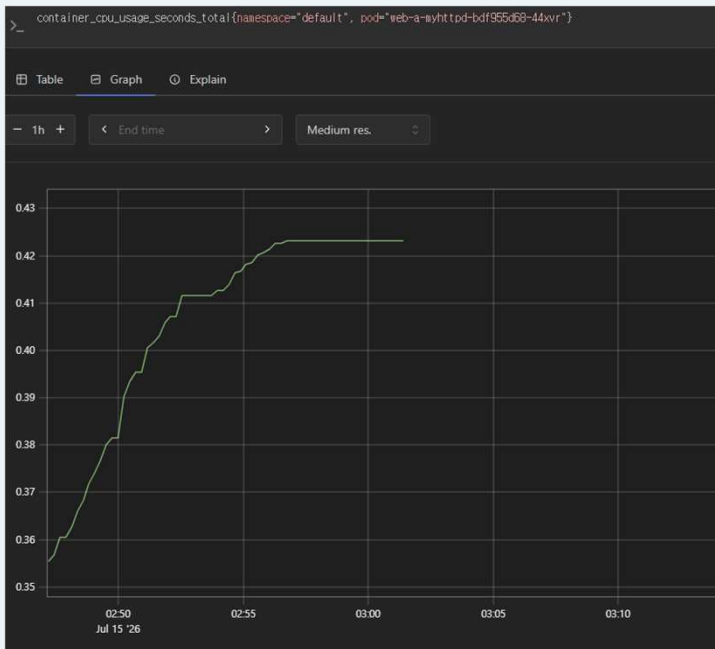
Prometheus, Grafana 캡처하기

프로젝트 2의 최종 결과물입니다



Prometheus와 Grafana 그래프 및 대시보드를 캡처하세요

생성한 Pod를 선택해 Prometheus의 그래프와, Grafana의 대시보드를 확인해주세요.



종합 체크리스트

두 프로젝트를 스스로 점검합니다

확인 항목	방법
Docker 이미지가 만들어졌는지	docker images 에서 ex) myapp:1.0 확인
Docker Hub에 올라갔는지	hub.docker.com에서 본인 리포지토리 확인 (캡처 완료)
Pod가 정상 배포됐는지	kubectl get pod
모니터링에서 보이는지	Prometheus, Grafana에서 그래프 및 대시보드 확인
최종 결과물	shjeon0708@gmail.com으로 이름과 프로젝트1, 2의 결과물을 스크린샷으로 제출